

Python Advanced

Generator Function

A generator function is a special type of function that returns an iterator object. Instead of using return to send back a single value, generator functions use yield to produce a series of results over time. The function pauses its execution after yield, maintaining its state between iterations.

Why Do We Need Generators?

- **Memory Efficient:** Handle large or infinite data without loading everything into memory.
- **No List Overhead:** Yield items one by one, avoiding full list creation.
- **Lazy Evaluation:** Compute values only when needed, improving performance.
- **Support Infinite Sequences:** Ideal for generating unbounded data like Fibonacci series.

The yield Keyword

The yield keyword is what makes a function a generator. When yield is encountered, the function's state is saved, and the value is returned. The next time the generator is called, it continues from where it left off.

Syntax:

```
def generator_function():  
    yield value
```

Example 1: Simple Generator

```
def count_up_to(n):  
    count = 1  
    while count <= n:  
        yield count  
        count += 1
```

```
gen = count_up_to(5)
```

```
for num in gen:  
    print(num)
```

Output:

```
1  
2  
3  
4  
5
```

Example 2: Using next()

```
def numbers():  
    for i in range(3):  
        yield i
```

```
gen = numbers()  
print(next(gen))  
print(next(gen))  
print(next(gen))
```

Output:

```
0  
1  
2
```

Example 3: Generator Expression

```
squares = (x*x for x in range(5))
```

```
for square in squares:  
    print(square)
```

Output:

```
0  
1  
4  
9  
16
```

ASSIGNMENTS

1. Write a generator function to generate odd numbers up to N.
2. Create a generator that produces the Fibonacci sequence up to N terms.
3. Write a generator that generates natural numbers infinitely. Display only the first 20 numbers using next().
4. Create a generator that reads a text file and yields one line at a time and after pressing the enter key the next line and so on.